Synthetic Data Compilation for Rollout-Free Trajectory Planning

Thomas Y.L. Lin [1]

Abstract

We revisit model-based reinforcement learning and remove recurrent rollout for downstream planning. Current action-conditioned models confront an efficiency-accuracy tradeoff for prediction: learning immediate next-state transitions requires fine temporal discretization and numerous rollout steps, while accurate regression needs massive trajectory data. We solve this tension by distilling a teacher representation of instantaneous dynamics into a student predictor. This gives both planning and training advantages. For planning, the student maps an action sequence to its consequences, avoiding recurrent rollout and backpropagation through a recurrent graph. For training, the teacher compiles synthetic supervision from any well-posed integration of the learned dynamics, improving generalization across trajectory distributions while preserving consistency with the governing law. We provide theoretical justification and empirical evidence for these advantages.

1 Introduction

We revisit model-based reinforcement learning (MBRL) and remove recurrent rollout for downstream planning. Our motivation comes from mainstream action-conditioned world models (Hafner et al., 2020; Hansen et al., 2022b) that map a current state and action to the immediate next state, implicitly assuming locally constant actions under zero-order hold (ZOH). Such modeling relies on fine temporal discretization and thus requires numerous recurrent rollout steps for planning. For example, with $\Delta t \approx 0.02$s, a 10-second query requires 500 rollout steps (Tassa et al., 2018; Hafner et al., 2020; Hansen et al., 2022b). Moreover, action planning through this recurrent graph becomes costly and susceptible to vanishing or exploding gradients (Pascanu et al., 2013).

Removing recurrent rollout suggests learning an endpoint predictor that maps an initial state and an action sequence directly to its consequence. However, direct endpoint regression is high-dimensional, so trajectory data coverage becomes sparse as the action-sequence length grows. Together, rollout-based modeling and endpoint prediction expose a tradeoff between planning cost and large-scale supervision.

We resolve this tradeoff by compiling synthetic data from a learned teacher dynamics model. The teacher predicts the instantaneous state derivative from local state-action pairs, i.e. $\dot{z} = f(z, u)$. For each supported action sequence, we integrate the teacher along the induced trajectory and use the resulting endpoint to distill a student predictor.

Advantages. The proposed method (Figure 1) shows three advantages:

- **Remove Planning Rollout.** The endpoint predictor replaces recurrent rollout with direct consequence query. When the reward objective uses K checkpoints (9), the model evaluates K endpoint queries that can be batched in parallel rather than sequentially (Section 3).
- **Direct Action Gradients.** Planning through the predictor optimizes the action sequence by backpropagating through a single network instead of a recurrent one. This removes products of Jacobians and gradient sensitivity in rollout planning (Lemma 3.1 and Corollary 3.2).
- **Synthetic Endpoint Supervision.** We compile synthetic endpoint targets by integrating the teacher along supported induced trajectories. This improves coverage across high-dimensional trajectory distributions while preserving consistency with the governing law (Section 4 and Proposition 4.1).

Roadmap. Section 2 formalizes. Sections 3 and 4 justify the advantages. Sections B and 5 report empirical tests. Section 6 concludes. Section A reviews related work.

2 Endpoint World Model

Controlled dynamical law. For latent state $z(t) \in \mathbb{R}^d$ and control $u(t) \in \mathbb{R}^m$, assume the governing law f follows

$$\frac{\mathrm{d}z}{\mathrm{d}t} = f(z(t), u(t)).$$

Note this law is autonomous: all relevant exogenous variables are included in z or u, so f has no explicit time dependency. For any well-posed control signal $u_{[t,t+T]} := u(\cdot)|_{[t,t+T]}$, f induces an *endpoint operator* Φ_T^f mapping

[1]University of Washington, Seattle, USA. Correspondence to: Thomas Y.L. Lin <thermaus@uw.edu>.

Preprint. May 20, 2026.

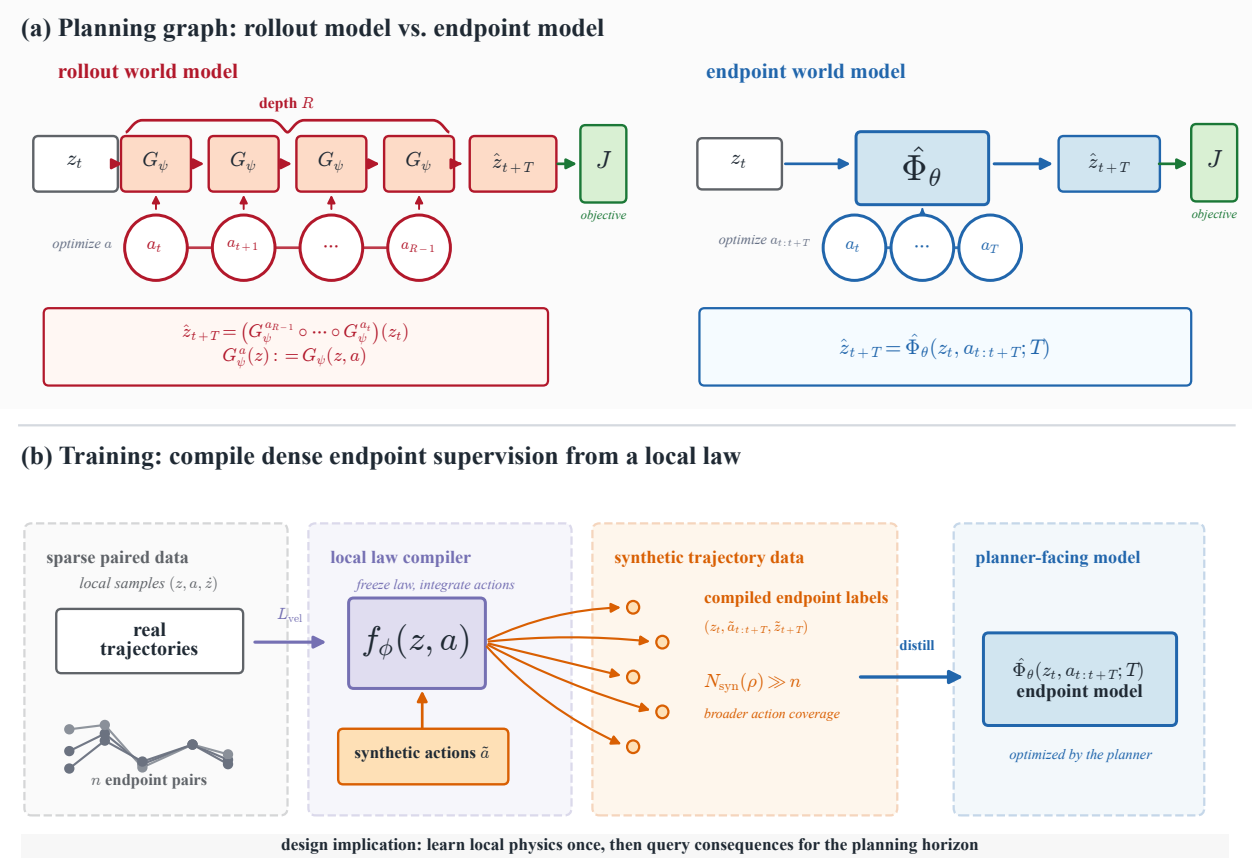

Figure 1. Method overview. **(a)** Planning. The rollout model G_ψ answers a horizon-T query by $R = T/\Delta t$ sequential calls; its action gradient is a product of R Jacobians and is prone to vanishing/exploding (Lemma 3.1). The endpoint world model $\hat{\Phi}_\theta$ maps the initial state and the full action sequence to the endpoint in a single forward pass, so the planner backpropagates through one network Jacobian. The semigroup identity lets the controller compose $\hat{\Phi}_\theta$ at a tunable inference depth K_{inf}. **(b)** Two-stage training. Stage 1 learns a dynamics law f_ϕ from real trajectories by velocity matching. Stage 2 integrates the frozen f_ϕ along synthetic action sequences to compile endpoint targets, which distill the student $\hat{\Phi}_\theta$; the synthetic supervision count $N_{\text{syn}}(\rho) \gg n$ exceeds the paired-data count.

state $z_t := z(t)$ to future state z_{t+T} satisfying

$$\Phi_T^f(z_t, u_{[t,t+T]}) = z_t + \int_t^{t+T} f(z(\tau), u(\tau))\, \mathrm{d}\tau. \quad (1)$$

A result-driven planner searches for a control signal whose predicted endpoint satisfies the desired condition.

Endpoint World Model. We propose learning a horizon-conditioned proxy for the endpoint operator:

$$\hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) = \hat{z}_{t+T}, \quad (2)$$

where $\hat{z}_{t+T}$ denotes an estimate of z_{t+T}. Practically, the control signal $u_{[t,t+T]}$ is discretized as an action sequence $a_{0:R-1} \in \mathbb{R}^{Rm}$. Under zero-order hold (ZOH), $u(\tau) = a_j$ for $\tau \in [t + j\Delta t,\ t + (j+1)\Delta t)$ with $\Delta t = T/R$. Splines and basis expansions give alternative finite-dimensional parameterizations of the same control signal (Hargraves & Paris, 1987; Kelly, 2017; Ijspeert et al., 2013).

Law-first compilation. We first learn the control law f_ϕ (Chen et al., 2018; Yildiz et al., 2021), then train the endpoint model (2) to match the endpoint operator (1)

$$\Phi_T^{f_\phi}(z_t, u_{[t,t+T]}) := z_t + \int_t^{t+T} f_\phi(z(\tau), u(\tau))\, \mathrm{d}\tau,$$

where the distillation loss (Salimans & Ho, 2022) is

$$\mathcal{L}_{\text{distill}} = \left\| \hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) - \Phi_T^{f_\phi}(z_t, u_{[t,t+T]}) \right\|^2. \quad (3)$$

Since f_ϕ maps local state-control pairs to instantaneous state changes, it can be integrated along any well-posed queried control signal $\tilde{u}_{[t,t+T]}$, whether observed, synthetic, or recombined. Each such query defines a teacher target $\tilde{z}_{t+T} = \Phi_T^{f_\phi}(z_t, \tilde{u}_{[t,t+T]})$, yielding dense supervision beyond the finite paired trajectory dataset.

Optional real-data anchoring. When paired samples $(z_t, u_{[t,t+T]}, T, z_{t+T})$ are available, we regularize $\hat{\Phi}_\theta$ to-

ward the observed true endpoint z_{t+T} rather than the synthetic $\Phi_T^{f_\phi}$ induced by f_ϕ (Song & Dhariwal, 2024):

$$\mathcal{L}_{\text{anchor}} = \left\| \hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) - z_{t+T} \right\|^2. \tag{4}$$

The synthetic targets from f_ϕ supply dense coverage over states, horizons, and control signals, while the real-data anchors accuracy on the support of the paired dataset. Direct endpoint regression is the special case where only this term is used and introduced as a baseline below at (7) .

Semigroup-consistency regularization. The endpoint operator induced by any controlled law satisfies the semigroup identity $\Phi_T^f = \Phi_{T-s}^f \circ \Phi_s^f$ for any $0 \le s \le T$. When $\hat{\Phi}_\theta$ is queried at intermediate sub-horizons it should compose consistently with its own one-call prediction. We encode this as an auxiliary loss:

$$\mathcal{L}_{\text{sg}} = \left\| \hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) - \hat{\Phi}_\theta\Big(\hat{\Phi}_\theta(z_t, u_{[t,t+s]}; s),\ u_{[t+s,t+T]}; T-s \Big) \right\|^2, \tag{5}$$

with $s \sim \text{Unif}(0, T)$ and T drawn from the same distribution as the distillation loss. This regularizer is optional but useful when the student is trained on a small set of horizon anchors and we want consistent predictions at intermediate horizons not in that set; it also enables the controller to invoke $\hat{\Phi}_\theta$ at variable depth (Section 3).

Average-velocity parameterization. Alternatively, we parameterize the endpoint model as $\hat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) = z_t + T\,\bar{v}_\theta(z_t, u_{[t,t+T]}; T)$, so the network outputs the average velocity rather than the endpoint itself (Geng et al., 2026). The distillation loss (3) rescales by $1/T$ to $\bar{v}_\theta(z_t, u_{[t,t+T]}; T) \approx \frac{1}{T}\int_t^{t+T} f_\phi(z(\tau), u(\tau))\,\mathrm{d}\tau$. This enforces the identity $\hat{\Phi}_\theta(z_t, u_{[t,t+T]}; 0) = z_t$ and keeps the output well-scaled in the small-horizon limit rather than dominated by z_t.

Reference baselines. We justify our proposed world model $\hat{\Phi}_\theta$ against two reference models, one for planning (Section 3) and one for training (Section 4).

The first is a *rollout model* representing local transition (Hafner et al., 2020; Hansen et al., 2022a) $G_\psi(z_t, a_t) \approx z_{t+\Delta t}$. It answers the planner's finite-horizon query by rolling out for $R = T/\Delta t$ steps:

$$\hat{z}_{t+T} = \big(G_\psi(\,\cdot\,, a_{t+(R-1)\Delta t}) \circ \cdots \circ G_\psi(\,\cdot\,, a_t)\big)(z_t). \tag{6}$$

The second is an *endpoint regressor*, which shares the endpoint supervision but is directly trained on paired trajectory samples $(z_t, u_{[t,t+T]}, T, z_{t+T})$

$$\mathcal{L}_{\text{end}} = \left\| G_\gamma(z_t, u_{[t,t+T]}; T) - z_{t+T} \right\|^2. \tag{7}$$

3 Planning Advantage

Given a world model, a planner queries predicted future states $\hat{z}_\tau$ to evaluate candidate control signals. Let $r(z, u)$ be an instantaneous reward and $g(z_T)$ a terminal reward. The planner solves the functional optimization

$$\max_{u_{[t,t+T]}} \int_t^{t+T} r\left(\hat{z}(\tau), u(\tau)\right) \mathrm{d}\tau + g\left(\hat{z}_{t+T}\right), \tag{8}$$

to search for the optimal $u_{[t,t+T]}$. Discretize the candidate control signal at the dynamics rate as $a_{0:R-1} \in \mathbb{R}^{Rm}$, where $R = T/\Delta t$. The objective, however, samples only $K \ll R$ reward checkpoints $\tau_j = t + jT/K$, using timestep weight $\Delta\tau = T/K$ and checkpoint action $\bar{a}_j := a_{\lfloor jR/K \rfloor}$. The optimization (8) then becomes finite-dimensional:

$$\max_{a_{0:R-1}} \sum_{j=0}^{K-1} r\left(\hat{z}_{\tau_j}, \bar{a}_j\right) \Delta\tau + g\left(\hat{z}_{t+T}\right). \tag{9}$$

The planning cost has two factors: (i) how expensively the world model answers each checkpoint query $\hat{z}_{\tau_j}$, and (ii) how large the search space over control signals $u_{[t,t+T]}$ is.

Query Cost. For a single query $\hat{z}_{\tau_j}$, the endpoint model evaluates $\hat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)$ once, regardless of the horizon length. The rollout model $G_\psi(z_t, a_t) \approx z_{t+\Delta t}$ returns the same query by $R_j = (\tau_j - t)/\Delta t$ nested calls,

$$\hat{z}_{\tau_j} = \big(G_\psi(\cdot, a_{R_j-1}) \circ \cdots \circ G_\psi(\cdot, a_0)\big)(z_t). \tag{10}$$

For each candidate control signal, the discrete planner in (9) reads K checkpoints $\hat{z}_{\tau_j}$. The rollout model caches intermediate states along the way, so it amortizes to R calls per candidate. The endpoint model directly queries the K required checkpoints. The aggregate per-candidate saving is $R - K$, recovering $R - 1$ in the terminal-only case $K = 1$.

The dynamics rate Δt must be small enough that G_ψ is Markovian and locally learnable at the underlying frame, motor-command, or decision rate. On the dm_control suite this gives $\Delta t \approx 0.02$s under the standard action-repeat-2 configuration also used by Dreamer and TD-MPC (Tassa et al., 2018; Hafner et al., 2020; Hansen et al., 2022a), so an episode of $T = 20$s already has $R = 10^3$. The number of checkpoints K, however, is set by where reward arrives. Most tasks of interest, e.g., Atari games (Mnih et al., 2015; Bellemare et al., 2013), board games (Schrittwieser et al., 2020), and goal-conditioned robotic manipulation (Andrychowicz et al., 2017), give reward only at score events or task completion, or $K \approx 1$. Hence the saving is essentially the entire rollout length.

Search Structure. For comparison, suppose both models are evaluated on the same grid-level action sequence $a_{0:R-1}$. Let $J_{\hat{\Phi}}(a_{0:R-1})$ denote the planner's objective (9) under the endpoint model. It depends on this action sequence through one-call queries $\hat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)$. Let

$J_G(a_{0:R-1})$ denote the same objective under the rollout model. It depends on the action sequence through the depth-R composition (10). Below we show the search structure of $J_{\widehat{\Phi}}(a_{0:R-1})$.

Lemma 3.1 (Rollout sensitivities contain sequential Jacobian products)**.** *Let the rollout be* $\hat{z}_{t+(r+1)\Delta t} = G_\psi(\hat{z}_{t+r\Delta t}, a_r)$ *for* $r = [0, R_j - 1]$ *and* $R_j = (\tau_j - t)/\Delta t$. *Assume* G_ψ *is differentiable. For any* $s < R_j$, *let* $A_\ell := \partial_z G_\psi(\hat{z}_{t+\ell\Delta t}, a_\ell)$ *and* $B_s := \partial_a G_\psi(\hat{z}_{t+s\Delta t}, a_s)$, *then*

$$\frac{\partial \hat{z}_{\tau_j}}{\partial a_s} = A_{R_j-1} A_{R_j-2} \cdots A_{s+1} B_s. \tag{11}$$

Proof. The identity follows by repeated application of the chain rule to the rollout recurrence. □

The product gives horizon-dependent sensitivity bounds. Let $d_{s,j} := R_j - s - 1$ be the number of intervening transition Jacobians $\{A_\ell\}_{\ell=s+1}^{R_j-1}$ between action a_s and checkpoint τ_j. If $\|A_\ell\| \le L_G$ and $\|B_s\| \le B_G$ along the rollout,

$$\left\| \frac{\partial \hat{z}_{\tau_j}}{\partial a_s} \right\| \le B_G L_G^{d_{s,j}}.$$

Thus, when $L_G < 1$, sensitivities to earlier actions decay exponentially with the number of intervening rollout steps. Conversely, if $\sigma_{\min}(A_\ell) \ge \mu_G$ and $\sigma_{\min}(B_s) \ge b_G$, then

$$\sigma_{\min}\left(\frac{\partial \hat{z}_{\tau_j}}{\partial a_s}\right) \ge \left(\prod_{\ell=s+1}^{R_j-1} \sigma_{\min}(A_\ell)\right) \sigma_{\min}(B_s) \ge b_G \mu_G^{d_{s,j}}.$$

So, if all intervening A_ℓ satisfy $\sigma_{\min}(A_\ell) \ge \mu_G > 1$, the sensitivity lower bound grows exponentially with depth.

Corollary 3.2 (Rollout planning requires summing all Jacobian-products)**.** *Suppose the rollout objective* J_G *follows:*

$$J_G(a_{0:R-1}) = \mathcal{J}\left(\hat{z}_{\tau_1}(a_{0:R-1}), \ldots, \hat{z}_{\tau_K}(a_{0:R-1}), a_{0:R-1}\right).$$

Then, for any action a_s,

$$\nabla_{a_s} J_G = \partial_{a_s} \mathcal{J} + \sum_{\substack{j=1 \\ s<R_j}}^{K} \left(\frac{\partial \hat{z}_{\tau_j}}{\partial a_s}\right)^\top \nabla_{\hat{z}_{\tau_j}} \mathcal{J},$$

where each checkpoint sensitivity $\partial \hat{z}_{\tau_j}/\partial a_s$ *requires calculating the sequential Jacobian-products in Lemma 3.1.*

Proof. Apply the chain rule to the composition of $\mathcal{J}$ with the rollout-generated checkpoints. □

For any action a_s preceding a queried checkpoint τ_j, i.e., $s < R_j$, Lemma 3.1 shows that $\partial \hat{z}_{\tau_j}/\partial a_s$ contains a product of $R_j - s - 1$ transition Jacobians. Corollary 3.2 further shows that gradients of the rollout objective J_G sum over these checkpoint sensitivities. This is the same failure mode as in recurrent networks (Pascanu et al., 2013; Metz et al., 2021); related mitigations are discussed in Section A.

The endpoint objective bypasses this online recurrent graph by querying each checkpoint directly as $\widehat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)$. Each checkpoint gradient differentiates through one network Jacobian rather than R_j sequential Jacobians (11). Meanwhile, the K endpoint queries $\{\widehat{\Phi}_\theta(z_t, u_{[t,\tau_j]}, \tau_j - t)\}_{j=1}^K$ share the broadcast input $\mathbf{1}_K \otimes z_t$ and can be resolved in one parallel forward pass. Together, these compress the rollout planner's gradient graph in both depth and width.

Depth-K_{inf} inference. The semigroup identity $\Phi_T^f = \Phi_{T-s}^f \circ \Phi_s^f$ gives a continuum of inference depths $K_{\text{inf}} \in [1, R]$ between the depth-1 endpoint query and the depth-R rollout (10). For any partition $T = \sum_{k=1}^{K_{\text{inf}}} \tau_k$,

$$\widehat{\Phi}_\theta(z_t, u_{[t,t+T]}; T) \approx \widehat{\Phi}_\theta(\cdot\,; \tau_{K_{\text{inf}}}) \circ \cdots \circ \widehat{\Phi}_\theta(z_t, u; \tau_1), \tag{12}$$

exact under the semigroup identity and tightened by $\mathcal{L}_{\text{sg}}$ (5). Each call is itself depth-1, so the gradient graph of (12) has K_{inf} Jacobian terms instead of R, and Lemma 3.1 applies with $\prod_{\ell=k+1}^{K_{\text{inf}}} A_\ell$ rather than $\prod_{\ell=s+1}^{R-1} A_\ell$. For $K_{\text{inf}} \ll R$ the planner keeps the depth-1 conditioning advantage.

Two implications follow. *(i) Inference-time compute scaling*: the same network supports any $K_{\text{inf}} \in [1, R]$ at $K_{\text{inf}} \cdot G$ forward calls per gradient plan, dialed at the controller. *(ii) Training-scope decoupling*: each leg only needs accuracy at its sub-horizon $\tau_k \le T_{\text{train}}$, so longer planning horizons are reached by composition; the student need not be trained on horizons it will eventually be asked to plan at. Sub-goal and waypoint planners (Section A) share the depth-1-per-stage property of (12), while rollout-within-stage hierarchies (TD-MPC's H-step lookahead (Hansen et al., 2022b), Dreamer's imagined rollouts (Hafner et al., 2020)) retain the depth-k Jacobian-product pathology inside each stage; $\widehat{\Phi}_\theta$ is a drop-in replacement that collapses each rollout into a single endpoint query. Section 5.1 (Table 2 and Figure 2) shows the empirical Pareto: the optimal K_{inf} grows with T but stays in single digits.

4 Training Advantage

Both direct endpoint regression and law-first compilation share the same supervision form. They differ in the target distribution. For $\Delta t_i = T_i/R_i$, let the trajectory dataset be

$$\mathcal{D}_{\text{traj}} = \{(z_0^{(i)}, a_0^{(i)}, z_1^{(i)}, \ldots, a_{R_i-1}^{(i)}, z_{R_i}^{(i)}, T_i)\}_{i=1}^n.$$

Direct endpoint regression fits $G_\gamma(z_0^{(i)}, a_{0:R_i-1}^{(i)}; T_i)$ to the observed endpoint $z_{R_i}^{(i)}$ via the tuples $(z_0^{(i)}, a_{0:R_i-1}^{(i)}, T_i, z_{R_i}^{(i)})$. Such supervision must cover the high-dimensional query space of states, horizons, and action sequences.

Law-first compilation generates supervision by integrating a learned control law f_ϕ (1). The law is trained on the $n_{\text{loc}} = \sum_{i=1}^n R_i$ samples $\{(z_r^{(i)}, a_r^{(i)}) \mapsto (z_{r+1}^{(i)} - z_r^{(i)})/\Delta t_i\}_{i,r}$.

Given f_ϕ, any well-posed action sequence $\tilde{a}_{0:R_i-1}$ defines a control signal $\tilde{u}_{[0,T_i]}$ and hence a teacher target $\tilde{z}_{R_i} = \Phi_{T_i}^{f_\phi}(z_0^{(i)}, \tilde{u}_{[0,T_i]})$. This target trains the student $\hat{\Phi}_\theta$ via distillation (3). The queried action sequence $\tilde{a}_{0:R_i-1}$ may be observed, synthetic, or recombined, and need not be paired with an observed endpoint. In the following paragraphs, we show law-first compilation dominates direct endpoint regression on $\mathcal{D}_{\text{traj}}$ in two ways: (i) expanding the effective supervision and (ii) shrinking the hypothesis class.

Scale of Synthetic Supervision We now quantify the synthetic supervision for $\hat{\Phi}_\theta$. Let the local training support be $\mathcal{D}_{\text{loc}} = \{(z_r^{(i)}, a_r^{(i)}) : i = 1, \dots, n\}_{r=0}^{R_i-1}$.

Proposition 4.1 (Synthetic supervision count)**.** *Fix $\rho > 0$. For state z, let $\mathcal{A}_\rho(z) := \{a : \text{dist}((z,a), \mathcal{D}_{\text{loc}}) \le \rho\}$. For anchor $z_0^{(i)}$, let $\tilde{z}_0 := z_0^{(i)}$ and $\tilde{z}_{r+1} := \tilde{z}_r + \Delta t_i\, f_\phi(\tilde{z}_r, \tilde{a}_r)$. The usable number of synthetic endpoint queries is $N_{\text{syn}}(\rho) := \sum_{i=1}^n |\mathcal{V}_\rho^{(i)}|$, where $\mathcal{V}_\rho^{(i)}$ is a valid action sequence set defined by*

$$\mathcal{V}_\rho^{(i)} := \left\{\tilde{a}_{0:R_i-1} : \tilde{a}_r \in \mathcal{A}_\rho(\tilde{z}_r)\ \forall r = 0, \dots, R_i - 1\right\}.$$

An action sequence is valid as each rollout pair $(\tilde{z}_r, \tilde{a}_r)$ stays within ρ of local training support, so f_ϕ generalizes along the synthetic trajectory and the endpoint target $\tilde{z}_{R_i}$ is reliable. The valid sets $\{\mathcal{V}_\rho^{(i)}\}_{i=1}^n$ induce a mixture over action sequences around the observed anchors. For example, sample an anchor $i \sim \text{Unif}(\{1, \dots, n\})$, then sample $\tilde{a}_{0:R_i-1}$ from a distribution supported on $\mathcal{V}_\rho^{(i)}$. Thus $N_{\text{syn}}(\rho)$ measures the size of the upsampled support for synthetic supervision. A ρ-net argument (Vershynin, 2018) on $\mathcal{A}_\rho$ gives $|\mathcal{V}_\rho^{(i)}| \gtrsim (\rho/\epsilon)^{d_a R_i}$ at resolution ϵ, hence $N_{\text{syn}}(\rho)$ grows exponentially in horizon and $N_{\text{syn}}(\rho) \gg n$ whenever $\rho > \epsilon$.

Smaller Hypothesis Class The student endpoint model $\hat{\Phi}_\theta$ distilled from the local teacher f_ϕ shares the same architecture and query space with the endpoint regressor G_γ. The statistical difference originates from the target. Denote $G_\gamma^{a,b}(\cdot) := G_\gamma(\cdot, u_{[a,b]}; b-a)$. Direct endpoint regression learns an unconstrained map $G_\gamma^{t,t+T}(z_t) \approx z_{t+T}$, but the loss does not enforce the semigroup identity

$$G_\gamma^{t,t+T} \not\equiv G_\gamma^{s,t+T} \circ G_\gamma^{t,s}, \qquad t < s < t+T.$$

In contrast, law-first compilation generates targets by integrating the learned local f_ϕ. So all teacher labels satisfy

$$\Phi_T^{f_\phi} = \Phi_{t+T-s}^{f_\phi} \circ \Phi_{s-t}^{f_\phi}, \qquad t < s < t+T.$$

Thus compilation supplies supervision constrained to a semigroup-consistent target family.

Let $\mathcal{H}_{\text{end}}$ be the hypothesis class of general horizon-conditioned endpoint maps and $\mathcal{H}_{\text{flow}} = \{\Phi^f : f \in \mathcal{F}_{\text{dyn}}\}$ the subclass induced by f. Since integrating f imposes semigroup consistency, $\mathcal{H}_{\text{flow}} \subsetneq \mathcal{H}_{\text{end}}$. Thus law-first compilation restricts the teacher class to $\mathcal{H}_{\text{flow}}$ and trains the student $\hat{\Phi}_\theta$ on semigroup-constrained supervision.

Combining the expanded supervision with ($N_{\text{syn}}(\rho) \gg n$) and shrunken hypothesis class ($\mathcal{H}_{\text{flow}} \subsetneq \mathcal{H}_{\text{end}}$), standard excess-risk bounds (e.g., Rademacher / VC) for law-first compilation are $\mathcal{O}(\text{cap}(\mathcal{H}_{\text{flow}})/N_{\text{syn}}(\rho))$, strictly smaller than $\mathcal{O}(\text{cap}(\mathcal{H}_{\text{end}})/n)$ for direct endpoint regression.

5 Empirical Evidence

We evaluate on Pendulum-v1 (Brockman et al., 2016) with timestep $\Delta t = 0.02$ s. The training set contains 128 trajectories of 300 steps (6 s each); the test set contains 64 trajectories. Section 5.1 reports terminal-cost planning over a $T = 2$ s horizon; Section 5.2 and the appendix ablations report prediction MSE over $T = 5.12$ s (256 steps). We compare our endpoint model $\hat{\Phi}_\theta$ trained in (3), against an *endpoint regressor* G_γ (7) and a *rollout model* G_ψ fit at Δt and rolled out $R = T/\Delta t$ times (6). Architecture, hyperparameters, planning protocol, and ablations are deferred to Section B.

5.1 Planning compute vs. task quality

The planning task is to drive the pendulum to the upright equilibrium $(\theta, \dot{\theta}) = (0, 0)$ from 30 random initial states. We use $K = 8$ control chunks over the $T = 2$ s horizon and report median terminal cost C and the fraction of plans reaching $C < 2$ (success). Endpoint models optimize the action sequence by Adam gradient descent on the planning cost. The rollout model has no usable action gradients (Lemma 3.1), so we plan with cross-entropy method instead and report an additional gradient-descent ablation row (Section B.3).

Table 1. Planning compute and task quality at $T = 2$ s, $K = 8$. Compute counts world-model forward calls per plan. Sampling planners (CEM and MPPI) use 128 candidates $\times$ 4 iterations; gradient planners use 50 Adam steps. Every cell decomposes into a closed form in these parameters (Table 4). $\hat{\Phi}_\theta^*$ is a single-horizon specialist trained at $T = 2$ s exactly; $\hat{\Phi}_\theta$ is the multi-horizon student of Table 2, evaluated here at $T = 2$ s.

Model	Planner	Compute	Median C	Reach
Oracle (true dynamics)	CEM	4,096	0.89	80%
Oracle (true dynamics)	gradient	400	0.38	97%
Endpoint regressor G_γ^{scalar}	gradient	400	0.94	73%
Endpoint regressor G_γ^{seq}	gradient	50	5.26	13%
Rollout model G_ψ	CEM	49,152	3.74	47%
Rollout model G_ψ	MPPI	49,152	3.23	40%
Rollout model G_ψ	gradient	4,800	0.52	77%
Ours $\hat{\Phi}_\theta^*$	gradient	50	0.73	90%
Ours $\hat{\Phi}_\theta$	gradient	**50**	**0.50**	**90%**

Result (single horizon, Table 1). Our multi-horizon endpoint world model $\hat{\Phi}_\theta$ reaches median terminal cost 0.50 at 90% reach at $T = 2$ s using only 50 forward calls per plan: lower median than oracle CEM (0.89/80%, 82$\times$ more compute) and the rollout model's gradient planner (0.52/77%, 96$\times$ more compute), and strictly better than our own single-

Table 2. Multi-horizon planning at $T \in \{1, 2, 3, 4, 5\}$ s (anchor horizons; the five planning targets on which the student is directly trained) and $T \in \{1.5, 2.5\}$ s (non-anchor stress-test horizons on which the student is never directly supervised), $K = 8$. Cells: median terminal cost / reach rate ($C < 2$) over 30 episodes. Same planner protocol as Table 1. Compute is forward calls per plan; for G_ψ it scales as $R = T/\Delta t$ (gradient: $R \cdot G$; CEM/MPPI: $N_{\text{cand}} \cdot N_{\text{iter}} \cdot R$), constant otherwise for one-call endpoint models. The single network $\hat{\Phi}_\theta$ admits a tunable inference depth K_{inf} via the semigroup property (Equation (5)); we report two rows for it. The $K_{\text{inf}} = 1$ row uses single-call inference at constant 50 forward calls per plan (cheap-inference regime). The $K_{\text{inf}} = K^*$ row uses the Pareto-optimal depth per cell drawn from $K_{\text{inf}} \in \{1, 2, 4, 8\}$ (see depth-K Pareto sweep, Figure 2); the cell-level superscript denotes the chosen K_{inf} and per-cell compute is $K_{\text{inf}} \cdot 50$ forward calls per plan. The $T = 2$ s specialist $\hat{\Phi}^*_\theta$ is included for contrast. Baselines on the non-anchor horizons are reported when measured.

			Anchor horizons (in-distribution)					Non-anchor (stress test)	
Model	Planner	Compute	T=1	T=2	T=3	T=4	T=5	T=1.5	T=2.5
Oracle	CEM	4,096	1.90/60%	0.89/80%	2.63/37%	4.35/37%	5.60/27%	—	—
Oracle	gradient	400	0.96/77%	0.38/97%	0.15/100%	0.12/100%	0.17/100%	—	—
G_γ^{scalar}	gradient	400	1.43/67%	0.94/73%	0.90/70%	0.73/77%	0.67/87%	—	—
G_ψ	CEM	$\approx 25\text{k} \cdot T$	1.65/60%	3.74/47%	3.89/33%	3.59/30%	1.76/57%	—	—
G_ψ	MPPI	$\approx 25\text{k} \cdot T$	2.00/50%	3.23/40%	2.74/40%	1.83/53%	4.34/27%	—	—
G_ψ	gradient	$\approx 2.5\text{k} \cdot T$	0.89/80%	0.52/77%	0.36/70%	0.80/73%	1.26/63%	—	—
Ours $\hat{\Phi}^*_\theta$	gradient	50	6.49/0%	0.73/90%	4.77/30%	4.36/30%	4.33/27%	—	—
Ours $\hat{\Phi}_\theta$, K_{inf}=1	gradient	50	0.88/80%	0.40/100%	0.78/90%	1.02/80%	4.27/20%	1.14/60%	0.79/77%
Ours $\hat{\Phi}_\theta$, $K_{\text{inf}}=K^*$	gradient	**50–400**	**0.88/80%**[1]	**0.40/100%**[1]	**0.18/80%**[4]	**0.12/83%**[8]	**0.03/93%**[8]	1.14/60%[1]	0.79/77%[1]

T specialist $\hat{\Phi}^*_\theta$ (0.73/90%). The scalar regressor G_γ^{scalar} (0.94/73%) pays 8× our compute from chaining R scalar calls per Adam step and still misses on both metrics. The sequence-aware variant G_γ^{seq} shares our one-call interface and compute, yet trained on real observed endpoints alone it never breaks the planning gap ($C = 5.26$, 13% reach): synthetic-teacher distillation, not the sequence interface, is what enables planning. CEM and MPPI on G_ψ both miss the oracle floor at 49,152 forward calls, ruling out planner choice as the source of the gap (Lemma 3.1 and Corollary 3.2).

Result (multi-horizon, Table 2). The same network $\hat{\Phi}_\theta$ serves planning at every horizon $T \in \{1, 2, 3, 4, 5\}$ s with a tunable inference depth K_{inf} (Equation (5)). *Single-call* inference ($K_{\text{inf}} = 1$, 50 forward calls per plan, constant in T) dominates short and mid-range horizons on every metric among non-oracle methods: at $T = 1$ s it matches G_ψ's gradient planner on reach rate (80%) at 48× less compute, at $T = 2$ s it ties or beats every baseline including the oracle CEM (0.40/100%), and at $T = 3$ s it achieves the highest non-oracle median (0.78) with 90% reach. At long horizons ($T \geq 4$ s), single-call planning degrades, but the same network at the Pareto-optimal $K_{\text{inf}} = K^*$ (Figure 2) *matches the oracle planner on median*: at $T = 4$ s, depth-8 composition achieves median 0.12 — identical to the oracle gradient planner's 0.12 at the same 400-call compute, against which G_γ^{scalar} also pays 400 calls and lands at 0.73; at $T = 5$ s, depth-8 composition reaches median 0.03, 5× below the oracle's 0.17 on the same compute budget. The underlying mechanism is that each sub-horizon T/K_{inf} stays inside the student's training support, so every component call in the composition operates in its accurate regime (Figure 2).

By contrast, the single-T specialist $\hat{\Phi}^*_\theta$ trained at $T = 2$ s exactly has 0% reach at $T = 1$ s and degrades to $\leq 30\%$ reach at every other horizon — quantifying the cost of horizon specialization: one model per planning horizon would require five separate T-specific training runs and five checkpoints. Our recipe achieves smooth coverage across the full $T \in [1, 5]$ s range with one network (Section B.2). Across all seven planning horizons measured, including the two non-anchor horizons $T \in \{1.5, 2.5\}$ s used as a smoothness stress-test, the geometric-mean median terminal cost at $K_{\text{inf}} = 1$ is 0.98 and the mean reach rate is 72% — a smooth multi-horizon planner at constant compute, with an additional 2×-compute lever for difficult long horizons.

5.2 Data efficiency under scarcity

To test Proposition 4.1 empirically, we retrain each pipeline on 10% of training trajectories (13 of 128). We report held-out endpoint MSE on varying-control test sequences across $R \in \{4, 8, 16\}$.

Table 3. Endpoint MSE at 10% training data and scarcity ratio (10% MSE / 100% MSE); ratio ≈ 1 indicates no degradation.

	R=4	R=8	R=16
Endpoint MSE at 10% *training data*			
Endpoint regressor G_γ^{scalar}	321.90	220.18	2407.86
Endpoint regressor G_γ^{seq}	19.53	19.38	18.13
Rollout model G_ψ	12.81	11.33	11.31
Ours $\hat{\Phi}_\theta$	19.71	14.26	12.70
Scarcity ratio (10% MSE / 100% MSE)			
Endpoint regressor G_γ^{scalar}	16.47	11.67	90.97
Endpoint regressor G_γ^{seq}	1.04	1.03	1.01
Rollout model G_ψ	2.01	1.28	1.15
Ours $\hat{\Phi}_\theta$	1.67	1.18	1.03

Result. The scalar regressor G_γ^{scalar} must cover the full (z_t, u, T) query space from n paired samples and collapses at 10% data, with scarcity ratios of 11–91×. The rollout model G_ψ and ours both train on local information (one-step transitions for G_ψ, integrated f_ϕ for ours) and approach full-data accuracy at $R = 16$ (ratios 1.15 and 1.03), realizing $N_{\text{syn}}(\rho) \gg n$ inside the semigroup-consistent class $\mathcal{H}_{\text{flow}} \subsetneq \mathcal{H}_{\text{end}}$. The sequence variant G_γ^{seq} also keeps ratios near 1, but its real-data training distribution is the degenerate constant-action support: absolute MSE stays at ∼18–20, well above ∼11–13 for G_ψ and ours. G_ψ pays for its accuracy with 100× more forward calls per plan (6); ours keeps both.

6 Concluding Remarks

We present synthetic data compilation for rollout-free trajectory planning. By replacing recurrent rollout with endpoint consequence queries, the method improves planning cost and action-gradient structure (Section 3). By compiling endpoint targets from a learned dynamics teacher, it improves supervision coverage under paired-data scarcity (Section 4).

Experiments support both advantages (Section 5).

References

Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Pieter Abbeel, O., and Zaremba, W. Hindsight experience replay. *Advances in neural information processing systems*, 30, 2017.

Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.

Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

Chane-Sane, E., Schmid, C., and Laptev, I. Goal-conditioned reinforcement learning with imagined subgoals. In *International conference on machine learning*, pp. 1430–1440. PMLR, 2021.

Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018.

Finn, C. and Levine, S. Deep visual foresight for planning robot motion. In *2017 IEEE international conference on robotics and automation (ICRA)*, pp. 2786–2793. IEEE, 2017.

Geng, Z., Deng, M., Bai, X., Kolter, Z., and He, K. Mean flows for one-step generative modeling. *Advances in Neural Information Processing Systems*, 38:75460–75482, 2026.

Hafner, D., Lillicrap, T., Fischer, I., Villegas, R., Ha, D., Lee, H., and Davidson, J. Learning latent dynamics for planning from pixels. In *International conference on machine learning*, pp. 2555–2565. PMLR, 2019.

Hafner, D., Lillicrap, T., Ba, J., and Norouzi, M. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, 2022a.

Hansen, N., Wang, X., and Su, H. Temporal difference learning for model predictive control. *arXiv preprint arXiv:2203.04955*, 2022b.

Hargraves, C. R. and Paris, S. W. Direct trajectory optimization using nonlinear programming and collocation. *Journal of Guidance, Control, and Dynamics*, 10(4):338–342, 1987. doi: 10.2514/3.20223.

Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

Hou, X., Huang, X., and Perdikaris, P. Cfo: Learning continuous-time pde dynamics via flow-matched neural operators. *arXiv preprint arXiv:2512.05297*, 2025.

Ijspeert, A. J., Nakanishi, J., Hoffmann, H., Pastor, P., and Schaal, S. Dynamical movement primitives: Learning attractor models for motor behaviors. *Neural Computation*, 25(2):328–373, 2013. doi: 10.1162/NECO_a_00393.

Janner, M., Li, Q., and Levine, S. Offline reinforcement learning as one big sequence modeling problem. *Advances in neural information processing systems*, 34: 1273–1286, 2021.

Janner, M., Du, Y., Tenenbaum, J. B., and Levine, S. Planning with diffusion for flexible behavior synthesis. *arXiv preprint arXiv:2205.09991*, 2022.

Jurgenson, T., Avner, O., Groshev, E., and Tamar, A. Subgoal trees a framework for goal-based reinforcement learning. In *International conference on machine learning*, pp. 5020–5030. PMLR, 2020.

Kelly, M. An introduction to trajectory optimization: How to do your own direct collocation. *SIAM Review*, 59(4): 849–904, 2017. doi: 10.1137/16M1062569.

Korda, M. and Mezić, I. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, 2018.

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., and Anandkumar, A. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.

Lin, H., Xu, Y.-Y., Sun, Y., Zhang, Z., Li, Y.-C., Jia, C., Ye, J., Zhang, J., and Yu, Y. Any-step dynamics model improves future predictions for online and offline reinforcement learning. *arXiv preprint arXiv:2405.17031*, 2024.

Lipman, Y., Chen, R. T. Q., Ben-Hamu, H., Nickel, M., and Le, M. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023.

Liu, Q. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.

Lu, L., Jin, P., Pang, G., Zhang, Z., and Karniadakis, G. E. Learning nonlinear operators via deeponet based on the universal approximation theorem of operators. *Nature machine intelligence*, 3(3):218–229, 2021.

Metz, L., Freeman, C. D., Schoenholz, S. S., and Kachman, T. Gradients are not all you need. *arXiv preprint arXiv:2111.05803*, 2021.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.

Pascanu, R., Mikolov, T., and Bengio, Y. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pp. 1310–1318. Pmlr, 2013.

Salimans, T. and Ho, J. Progressive distillation for fast sampling of diffusion models. *arXiv preprint arXiv:2202.00512*, 2022.

Schrittwieser, J., Antonoglou, I., Hubert, T., Simonyan, K., Sifre, L., Schmitt, S., Guez, A., Lockhart, E., Hassabis, D., Graepel, T., et al. Mastering atari, go, chess and shogi by planning with a learned model. *Nature*, 588(7839): 604–609, 2020.

Song, Y. and Dhariwal, P. Improved techniques for training consistency models. In *International Conference on Learning Representations*, volume 2024, pp. 15078–15097, 2024.

Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020.

Tassa, Y., Doron, Y., Muldal, A., Erez, T., Li, Y., Casas, D. d. L., Budden, D., Abdolmaleki, A., Merel, J., Lefrancq, A., et al. Deepmind control suite. *arXiv preprint arXiv:1801.00690*, 2018.

Vershynin, R. *High-dimensional probability: An introduction with applications in data science*, volume 47. Cambridge university press, 2018.

Yildiz, C., Heinonen, M., and Lahdesmaki, H. Continuous-time model-based reinforcement learning. In *International Conference on Machine Learning*, 2021.

A Related Work and Discussion

Rollout-based MBRL and recurrent planning. Long-horizon rollout depth is a standard difficulty in model-based RL. Backpropagation through a recurrent computation graph produces products of transition Jacobians (Lemma 3.1), which can cause vanishing or exploding sensitivities (Pascanu et al., 2013). Existing world-model methods mitigate the recurrent rollout structure rather than remove it. PlaNet plans by CEM in a learned recurrent latent space, regularized by latent overshooting for multi-step consistency (Hafner et al., 2019). Dreamer learns actors and critics by backpropagating through imagined latent rollouts of a compact recurrent dynamics model (Hafner et al., 2020). TD-MPC truncates planning horizons with a learned terminal value, reducing rollout depth at planning time (Hansen et al., 2022b). All three keep recurrent composition in the model loop. Our formulation removes the recurrent rollout itself. For terminal or sparse-reward objectives, the planner queries $\widehat{\Phi}_\theta(z_t, u_{[t,t+\tau]}; \tau)$ directly instead of traversing the recurrent graph (Section 3)

Hierarchical and decomposed planning. Horizon decomposition has two structural patterns. Rollout-within-stage schemes shorten per-stage horizon but each stage is a multi-step rollout: TD-MPC's H-step lookahead (Hansen et al., 2022b), Dreamer's imagined rollouts (Hafner et al., 2020). These still incur the depth-k Jacobian-product pathology (Lemma 3.1) inside each stage. Depth-1-per-stage schemes instead replace each stage with a single-shot prediction, typically sub-goal or waypoint reasoners that bisect the horizon (Jurgenson et al., 2020; Chane-Sane et al., 2021). Our compositional inference (12) sits in the depth-1-per-stage family, distinguished by: each call ingests a full action sub-sequence rather than a goal state (continuous controls, end-to-end gradient); the semigroup loss (5) explicitly trains the network for composability; and K_{inf} is a continuous controller-time knob. Diffusion / consistency planners (Janner et al., 2022; Song et al., 2020) are also depth-1-per-stage but compose in generative time, not physical time — a complementary axis.

Dynamics and operator learning. A broad line of work learns dynamical laws or solution operators rather than fixed-step transition maps. Neural ODEs and continuous-time MBRL parameterize instantaneous vector fields instead of immediate next-state predictors (Chen et al., 2018; Yildiz et al., 2021). Koopman-based methods learn lifted linear predictors for nonlinear controlled dynamics and have been used inside MPC (Korda & Mezić, 2018). Neural operators, including DeepONet and Fourier neural operators, learn mappings between function spaces and amortize PDE solution operators across initial conditions, parameters, or forcing functions (Lu et al., 2021; Li et al., 2020). These works primarily target prediction, simulation, system identification, or control within a learned or partially specified dynamical system. We use dynamics as a learned representation of controlled state change without a known simulator or symbolic model. The learned local law can then be recombined across supported action sequences to compile finite-horizon endpoint supervision (Sections 2 and 4).

State-change modeling across generative and physical time. Diffusion and score-based models learn denoising scores or reverse-time dynamics and generate samples through iterative solvers over diffusion time (Song et al., 2020; Ho et al., 2020). Flow matching and rectified flow instead learn velocity fields along probability paths (Lipman et al., 2023; Liu, 2022). One-step flow models replace instantaneous velocity with average state change to reduce inference-time solver depth (Geng et al., 2026). Although these methods are usually formulated in generative time, related physical-time work applies flow-matching ideas to continuous-time PDE dynamics by fitting evolution fields rather than autoregressive transitions (Hou et al., 2025). Our work uses the same state-change viewpoint in controlled dynamics, but the output is a planner-facing endpoint predictor rather than a generative sampler (Section 2).

Trajectory prediction and supervision scarcity. Action-conditioned visual foresight, any-step dynamics, and trajectory-level generative models move beyond immediate next-state prediction by modeling longer-horizon futures or full trajectories (Finn & Levine, 2017; Lin et al., 2024; Janner et al., 2021; 2022). Statistically, these approaches learn maps or distributions over high-dimensional trajectory objects whose dimension grows with horizon and action-sequence length. Yet valid trajectories are highly structured: they are consequences of a local controlled law, not arbitrary state-action sequences. Directly fitting future or trajectory maps from paired trajectory data can therefore spend capacity on a large joint object rather than the lower-dimensional dynamical rule that generates it. Law-first compilation uses this structure explicitly by compiling supported synthetic endpoint targets from the learned local law, improving trajectory coverage while constraining targets to dynamics-induced consequences (Section 4 and Proposition 4.1).

B Experimental Setup and Ablations

This section contains architecture, training-recipe, planning-protocol, and ablation details deferred from Section 5. All experiments use the same Pendulum-v1 dataset of 128 training trajectories and 64 held-out test trajectories, each 300 steps

long. Trajectories are generated with a constant uniform control in $[-2, 2]$ and integration step $\Delta t = 0.02$ s. Test trajectories use a different random seed.

B.1 Models

Control law f_ϕ. A multilayer perceptron (MLP) with 4 layers of width 256 and SiLU activations, mapping $(z, u) \mapsto \dot{z}$ with no time input. Trained by velocity matching against 5-point central finite differences (`velocity_matching_central_diff_order: 5`) on the trajectory dataset; the central-difference target reduces the velocity-matching bias from $\mathcal{O}(\Delta t)$ to $\mathcal{O}(\Delta t^4)$. Integrated with RK4 over 8 substeps for distillation queries.

Endpoint regressor G_γ^{scalar}. A multilayer perceptron (MLP) with 4 layers of width 512 and SiLU activations, mapping $(z_t, u, h) \mapsto v$ in the average-velocity parameterization $\hat{z}_{t+h} = z_t + h\, v$. Trained on observed (z_t, u, h, z_{t+h}) tuples sampled from the training trajectories with h drawn uniformly from $[1, 250]$ environment steps. Assumes the control u is held constant over $[t, t + h]$.

Endpoint regressor G_γ^{seq}. Same attention architecture as $\hat{\Phi}_\theta$ below, but trained on real observed endpoint tuples $(z_t, u_{[t,t+T]}, T, z_{t+T})$ instead of synthetic teacher rollouts. Because the raw trajectories are constant-control, every observed action sequence is degenerate (K copies of one scalar action). Isolates the contribution of synthetic supervision from the contribution of the sequence interface.

Rollout model G_ψ. A multilayer perceptron (MLP) with 3 layers of width 128, mapping $(z_t, u) \mapsto \hat{z}_{t+\Delta t}$ at a fixed step $\Delta t = 0.02$ s. At query time the model is unrolled $R = T/\Delta t = 96$ chunk-aligned base steps under zero-order hold of the candidate control (Section B.3).

Our endpoint world model $\hat{\Phi}_\theta$. A Transformer encoder over the action-chunk tokens $\{(u_r, h_r)\}_{r=0}^{R-1}$ produces a context vector that is concatenated with the initial state z_t and passed to an average-velocity head with 4 layers of width 512. The encoder uses 2 self-attention blocks with hidden dimension 128, 4 heads, and feed-forward width 512. One forward pass produces the predicted endpoint $\hat{z}_{t+T}$ regardless of R.

B.2 Two-stage training

Stage 1 (law). f_ϕ is trained by 5-point central-difference velocity matching on the trajectory dataset. We use batch size 256, 6000 optimization steps, Adam with learning rate 10^{-3} and weight decay 10^{-6}, gradient clipping at 1.0.

Stage 2 ($\hat{\Phi}_\theta$). The student is trained against teacher endpoints $\Phi_T^{f_\phi}(z_t, \tilde{u}_{[t,t+T]})$ obtained by integrating the frozen f_ϕ chunk-by-chunk under synthetic action sequences. We do not use observed endpoints during stage 2. Batch size 256, 6000 optimization steps, Adam with learning rate $7 \cdot 10^{-4}$ and weight decay 10^{-6}, gradient clipping at 1.0.

For each training step, we sample a chunk count K uniformly from $\{2, 4, 8, 16\}$ and a segment count B uniformly from $[1, K]$. The synthetic sequence is piecewise constant in B consecutive segments of equal length. Each segment's control value is sampled independently from the training-data action pool. The total horizon is fixed to $T = 5.12$ s (256 env steps), so each chunk has duration T/K. The teacher integrates f_ϕ sequentially across the K chunks with 8 RK4 substeps per chunk to produce z_{t+T}.

$B = 1$ recovers the constant-control distribution of the original training trajectories. $B = K$ allows each chunk's control to differ. Drawing B uniformly per sample exposes the student to a continuum of smoothness levels. The integrated rollout stays near states where f_ϕ is accurate.

B.3 Planning protocol

Gradient planning (endpoint models). For an endpoint operator $\hat{\Phi}_\theta$ and the endpoint regressor G_γ, the action sequence $a = (a_0, \ldots, a_{R-1}) \in \mathbb{R}^{Rm}$ is a parameter tensor. Adam optimizes the terminal cost C for 50 steps at learning rate 0.1 with initial standard deviation 0.3. Each step performs one forward and one backward pass through the model. Our endpoint world model $\hat{\Phi}_\theta$ takes the full sequence in one call, so each step costs one forward call (50 total). The endpoint regressor G_γ chains R scalar calls per step ($R \cdot 50$ total).

CEM (rollout baseline). The rollout model G_ψ does not admit a usable action gradient in the recurrent form (Lemma 3.1). We plan with cross-entropy method: a per-chunk Gaussian over the R-chunk control sequence, 128 candidates per iteration, 4 iterations, elite fraction 0.1, initial standard deviation 1.0. Inside each chunk G_ψ unrolls $T/\Delta t = 100$ steps regardless of the planner's chunk count. Total forward calls per plan is candidates $\times$ iterations $\times$ (R micro-steps per candidate).

MPPI (rollout baseline). We also report a Model-Predictive Path Integral baseline for G_ψ. MPPI uses the same per-chunk Gaussian and the same 128×4 sampling budget as CEM, but updates the Gaussian by the exponentially-weighted mean and variance of the candidate costs with temperature $\lambda = 1.0$ rather than by the top-N elite mean. This is the standard reference planner in recent model-based RL work and rules out the possibility that the rollout-model wedge in Table 1 is an artifact of a weak CEM-style aggregation.

Gradient ablation on G_ψ. For the gradient-descent row of G_ψ in Table 1, we apply the same Adam recipe through the chained R-step rollout. This pays $R\cdot$ steps forward calls per plan and tests whether the gradient pathology of Lemma 3.1 is empirically severe.

Compute counts in Table 1 as a closed form. Forward calls per plan are fully determined by the planner's parameters and each model's per-evaluation cost. Let $G = 50$ be the gradient planner's Adam steps, $N_{\text{cand}} = 128$ and $N_{\text{iter}} = 4$ the CEM candidates per iteration and iteration count, $K = 8$ the number of control chunks, and $R = T/\Delta t = 96$ the env micro-steps over the aligned horizon ($T = 1.92$ s, $\Delta t = 0.02$ s, with $R = K \cdot \lceil (T/K)/\Delta t \rceil$). One "forward call" is one invocation of `model.predict`, regardless of internal sub-stepping.

A gradient planner makes one forward and one backward pass per Adam step, so per-step cost equals the number of model evaluations required to produce $\hat{z}_T$. Sequence-aware endpoint models ($\hat{\Phi}_\theta$ and G_γ^{seq}) produce $\hat{z}_T$ in one call; scalar endpoint regressors (G_γ^{scalar}) chain K calls; rollout models (G_ψ) chain R calls. CEM evaluates $N_{\text{cand}} \cdot N_{\text{iter}}$ candidates per plan, each requiring as many forward calls as the model needs to score the trajectory. The Oracle is counted at its API granularity (`env.step`(chunk_h) once per chunk).

Table 4. Compute count per plan as a closed form in the planner and model parameters. Substituting $K = 8$, $R = 96$, $G = 50$, $N_{\text{cand}}N_{\text{iter}} = 512$ reproduces every cell of Table 1.

Model	Planner	Forward calls / plan	Value
$\hat{\Phi}_\theta$ (Ours)	gradient	G	50
G_γ^{seq}	gradient	G	50
G_γ^{scalar}	gradient	$K \cdot G$	400
G_ψ	gradient	$R \cdot G$	4,800
Oracle (true dynamics)	gradient	$K \cdot G$	400
Oracle (true dynamics)	CEM	$N_{\text{cand}}N_{\text{iter}}K$	4,096
G_ψ	CEM	$N_{\text{cand}}N_{\text{iter}}R$	49,152
G_ψ	MPPI	$N_{\text{cand}}N_{\text{iter}}R$	49,152

The closed-form structure makes the compute comparisons in Table 1 reducible to two ratios of model evaluations: (i) *one-call vs. chained-scalar* at the gradient planner ($\hat{\Phi}_\theta$ vs. G_γ^{scalar}): G vs. $K \cdot G$, a factor of $K = 8$; (ii) *gradient vs. CEM* at the same dynamics (Oracle gradient vs. Oracle CEM): KG vs. $N_{\text{cand}}N_{\text{iter}}K$, a factor of $N_{\text{cand}}N_{\text{iter}}/G = 512/50 \approx 10.2$. Composing these, $\hat{\Phi}_\theta$ vs. Oracle CEM is $\approx 81.9\times$ — the headline ratio reported in Table 1. The first factor is the paper's contribution (the one-call sequence interface); the second is a known planner-paradigm effect that any gradient-friendly endpoint model would inherit.

Execution and aggregation. After planning, the chosen control sequence is executed on the true environment with zero-order hold inside each chunk. We record the terminal cost $C := \theta^2 + 0.1\,\dot{\theta}^2$ at $t = T$. The cost distribution over initial states is bimodal. Plans either reach the target or miss by a wide margin. We aggregate with the median rather than the mean, which would be dominated by the failed plans.

Depth-K_{inf} inference Pareto sweep. Figure 2 shows the per-T Pareto curves underlying the $K_{\text{inf}} = K^*$ row of Table 2. Each curve sweeps $K_{\text{inf}} \in \{1, 2, 4, 8\}$ for one fixed planning horizon T. The optimal K_{inf} grows with T (single-call wins at short T; depth-8 at long T), bounded by the constraint that the per-leg sub-horizon T/K_{inf} stay inside the student's training-anchor support.

B.4 Ablations

Smoothness control helps most at short chunk counts. At $R = 4$ the smoothness-controlled student has scarcity ratio 0.84 vs 1.06 for independent-uniform; the two converge near $R = 16$. Independent-uniform wins absolute MSE at 100% data but loses the data-efficiency advantage.

Sequence input matters at planning time. A scalar-input variant of the law-first student is distilled from f_ϕ but takes one action per call. Its absolute MSE and scarcity ratio match the sequence variants (Table 5, row 1) — distillation alone

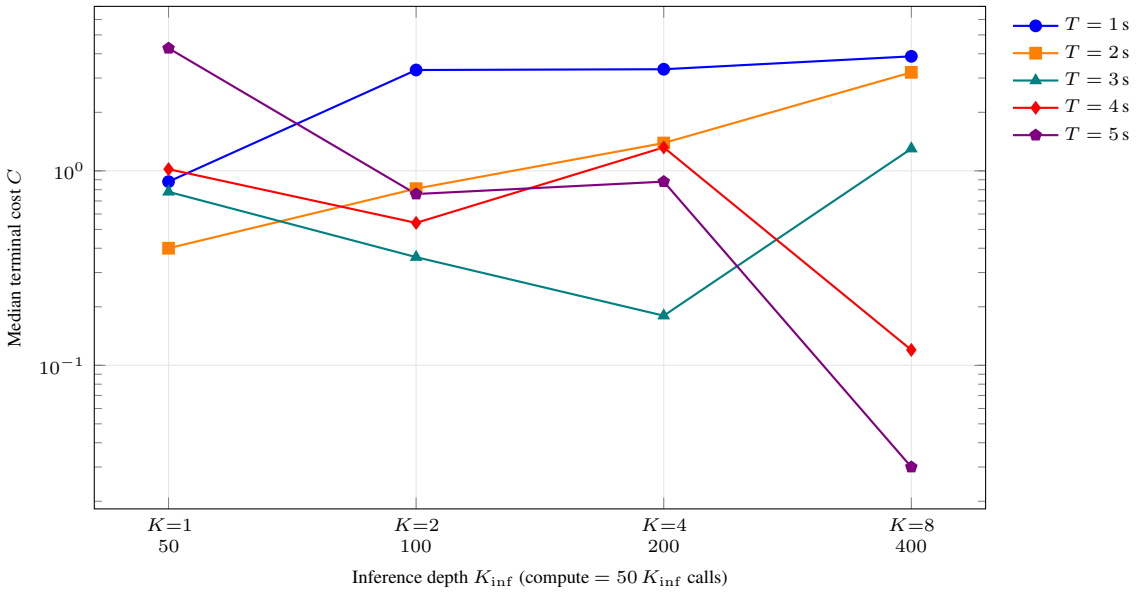

Figure 2. Depth-K_{inf} inference sweep for the multi-horizon student $\hat{\Phi}_\theta$ (a9-long). Each curve is one planning horizon T, swept over inference depths $K_{\mathrm{inf}} \in \{1, 2, 4, 8\}$ (compute $= 50\,K_{\mathrm{inf}}$ forward calls per plan). The Pareto-optimal K_{inf} grows with T: single-call best at $T \leq 2$ s; depth-8 at $T \in \{4, 5\}$ s (median C dips below 0.2). Composition hurts when the per-leg sub-horizon T/K_{inf} falls below the student's smallest training anchor (e.g. at $T = 1$ s, $K = 2$ already drops the leg to 0.5 s, below the 0.96 s anchor floor; performance degrades). One trained network underlies every point.

Table 5. Endpoint MSE at 20% training data, varying-control test sequences. Removing the smoothness control pushes the scarcity ratio above the scalar baseline.

Variant	R=4	R=8	R=16
Endpoint MSE at 100% *training data*			
Scalar distilled (no sequence input)	14.34	13.78	13.35
Sequence, independent-uniform synthetic	10.08	11.41	12.29
Sequence, smoothness-controlled (ours)	11.82	12.07	12.35
Scarcity ratio (20% MSE / 100% MSE)			
Scalar distilled (no sequence input)	1.02	1.08	1.04
Sequence, independent-uniform synthetic	1.06	1.09	1.02
Sequence, smoothness-controlled (ours)	**0.84**	**0.96**	**1.00**

protects data efficiency — but gradient planning costs $R\times$ more forward calls per Adam step than the one-call sequence student, losing the compute advantage of Section 5.1.

B.5 Generalization to unseen horizons

We test whether the two-stage construction transfers beyond the training horizon range. The first two rows hard-cap training data at $h \leq 50$ steps; the third keeps the same f_ϕ but trains the student on synthetic teacher rollouts up to $h \leq 256$, well beyond the raw-data limit. Predictions and ground truth use a single constant control held over the full horizon.

Table 6. Endpoint MSE at in- and out-of-distribution horizons. Training horizons are capped at $h = 50$. Entries marked * are unseen at training time.

Endpoint MSE	h=50	h=75*	h=100*
G_γ, $h \leq 50$ (raw-data limit)	**0.35**	6.13	13.78
Ours $\hat{\Phi}_\theta$, $h \leq 50$ synthetic	1.72	17.11	35.90
Ours $\hat{\Phi}_\theta$, $h \leq 256$ synthetic	15.74	**10.53**	**5.78**

At $h = 50$, G_γ fits best (MSE 0.35). The $h \leq 50$ student inherits the same cap and fails to generalize past it (MSE 17.1 at $h = 75$, 35.9 at $h = 100$). The $h \leq 256$ student is the only model whose error *decreases* past the raw-data limit, reaching MSE 5.78 at $h = 100$ (2.4$\times$ lower than G_γ). Horizon extrapolation requires using f_ϕ to manufacture supervision past the raw-data range.